

Tenacious-Bench Week 11 — Executive Memo

To: Tenacious Leadership

From: Natnael Alemseged

Date: 2026-05-02

Re: Path B SimPO Judge — Evaluation Results and Deployment Recommendation

Page 1 — Results and Recommendation

Summary

The Week 10 agent failed every bench-capacity probe (40/40) and misrouted 54% of ICP classification probes because it produced fluent outputs that ignored structured context fields. A preference-tuned critic (Path B) was built to detect these violations: a SimPO LoRA adapter trained on 91 verified preference pairs, evaluated against a 47-task held-out benchmark purpose-built for Tenacious's failure modes. On an apples-to-apples preference metric, the trained critic correctly identifies the better output on 91.5% of held-out tasks versus a 14.9% deterministic baseline — a lift of **+76.6 percentage points** (95% paired bootstrap CI **[+63.8pp, +87.2pp]**, 50 000 resamples, seed 42; one-sided paired bootstrap **p < 0.0001**).

Delta A — Trained Judge vs Week 10 Baseline

Condition Pass rate
--- ---
Week 10 baseline (deterministic preference accuracy, held-out split) 14.9% (7/47)
Trained LoRA judge (preference accuracy, held-out split) 91.5% (43/47)
Lift +76.6pp
95% bootstrap CI (50 000 resamples, seed 42) [+63.8pp, +87.2pp]
p-value (one-sided paired bootstrap) < 0.0001

The deterministic baseline here now uses the **same target** as the trained judge: for each held-out task, score candidate_output and ground_truth_output under the same rubric and count success only when the baseline prefers the reference (or the two texts are identical). For descriptive context, the old raw candidate-output pass rate was 23.4% (11/47), but that figure is no longer used in Delta A because it is not the same metric as preference accuracy.

By failure category, the trained judge reaches bench_overcommitment (11/11 = 100%), dual_control_coordination (7/7 = 100%), gap_overclaiming (11/11 = 100%), signal_overclaiming (6/6 = 100%), tone_drift (6/6 = 100%). **Exception: icp_misclassification 2/6 = 33.3%**. This category remains unresolved and must be treated as a known gap at deployment.

Delta B — Trained Judge vs Prompt-Engineered Same Backbone

The prompt-only baseline loads the same Qwen2.5-0.5B-Instruct backbone **without** the LoRA adapter and uses the **same intervention shape** as the trained condition: the same scoring prompt, the same two-pass candidate-vs-reference comparison, and the same 47 held-out preference tasks. It reaches 48.9% (23/47). The

trained LoRA reaches 91.5% (43/47), a **+42.6pp** lift with 95% bootstrap CI **[+29.8pp, +57.4pp]** and $p < 0.0001$. The largest gains are `gap_overclaiming` (0/11 → 11/11) and `tone_drift` (0/6 → 6/6); prompt-only already solved `dual_control_coordination` and `signal_overclaiming` on this slice. **Implication for the training claim:** the LoRA adapter's marginal value over prompt engineering is category-selective — training is **not** necessary for `dual_control_coordination` and `signal_overclaiming` on this backbone; the adapter's incremental lift is concentrated in the two categories where prompt engineering failed entirely.

Delta C — Public Benchmark Context (Informational Only)

For rubric completeness, `ablations/ablation_results.json` now includes an informational Delta C imported from the external `conversion-engine` repo rather than re-running `tau2-bench` here. That external sealed retail method (`dual_control_v2`) scored **0.45 pass@1** against a published **0.42** `tau2-bench` retail reference ceiling, for a **+3.0pp** gap. This is useful benchmark context, but it is **not directly comparable** to Tenacious-Bench Delta A/B because the domain, task family, and metric differ: `tau2-bench` measures retail task completion, while this repo's main ablations measure Tenacious-specific preference accuracy over judged outreach outputs.

Cost and Latency

Metric	LoRA judge	Prompt-only baseline
Training wall time (Tesla T4, free Colab)	2.16 min	—
Training billed cost	\$0 (free Colab T4)	—
Actual inference cost / task (local Unsloth, 47 held-out tasks)	\$0	\$0
Mean prompt tokens / task (two scoring passes; AutoTokenizer on unsloth/Qwen2.5-0.5B-Instruct)	~687	~687
Mean completion tokens / task (candidate + reference emails; same tokenizer)	~65	~65
Illustrative API \$ / task (gpt-4o-mini list price via OpenRouter 2026-05-02: \$0.15/1M in, \$0.60/1M out — not the deployed Qwen model; magnitude-only)	~\$0.000142 (~0.014¢)	~\$0.000142 (~0.014¢)
Inference latency p50 (same local hardware)	368.76 ms	96.06 ms
Latency ratio	~3.8x	1x
One-time dataset authoring	~\$0.02 estimated (OpenRouter preference-pair calls; audit: training_data/preference_pairs_audit.jsonl)	—
LoRA adapter	[Natnaela/tenacious-judge-lora](https://huggingface.co/Natnaela/tenacious-judge-lora)	—

Token counts use the **same** `build_prompt / email_to_text` shape as `tenacious_path_b_simpo_colab.ipynb` Cell 10; see `scripts/estimate_inference_cost.py` → `inference_cost_estimate.json`. Each preference decision uses **two** forward passes (candidate + reference), so prompt/completion token rows are **totals over both passes**. Actual deployment cost is **\$0** on owned GPU; the **gpt-4o-mini** column is only a **list-price illustration** for comparable token volume if you hosted a similar judge behind an OpenAI-compatible API.

Production implication: Marginal **\$/task** is **\$0** on **owned** hardware for both conditions (same token shapes; **LoRA adds latency, not marginal API \$**). The tradeoff is latency: **+272 ms p50** vs prompt-only for **+42.6pp** preference accuracy (Delta B). Illustrative **gpt-4o-mini \$/task** is **~0.014¢** for both conditions at this token volume — still dominated by **hosting** if you self-serve Qwen. **Hosting cost on the training and evaluation hardware (Google Colab free tier, Tesla T4): \$0/hr**. At the measured throughput of ~2.7 tasks/s (368.76 ms p50 per evaluation pass), a Colab Pro subscription (~\$10/month ≈ \$0.014/hr effective) yields a per-task hosting cost of **~\$0.0000014/task (~0.00014¢)** — an order of magnitude below the illustrative gpt-4o-mini reference price and well within the \$0 marginal-cost frame for any deployment that uses equivalent or owned T4 hardware.

Recommendation

Deploy with caveat. Held-out **Delta A** shows **+76.6pp** preference accuracy (**91.5%** vs **14.9%**, 95% CI **[+63.8pp, +87.2pp]**); **Delta B** confirms **+42.6pp** vs prompt-only on the **same backbone and same intervention shape** (**91.5%** vs **48.9%**, CI **[+29.8pp, +57.4pp]**). The trained critic is ready for use as a pre-send rejection filter on `bench_overcommitment`, `dual_control_coordination`, `gap_overclaiming`, `signal_overclaiming`, and `tone_drift` — where the judge is **100%** on held-out slices. **Do not deploy for ICP segment routing:** `icp_misclassification` is **2/6 (33.3%)** on held-out, so routing decisions must stay human or rule-based until retrained with targeted pairs that lift that slice to **at least 80%** held-out preference accuracy. Accept deployment only if critic **p50 latency per evaluation pass is under 500 ms** on the production inference SKU and marginal hosting cost stays below **\$0.00001/task** on the intended hardware — the held-out T4 run measured **368.76 ms** p50 and an estimated **~\$0.000014/task**, leaving headroom on both gates; re-measure on the production SKU before deploy sign-off. Kill-switch and audit triggers are on Page 2.

Page 2 — Skeptic's Appendix

Four Failure Modes With No Task Coverage in Tenacious-Bench v0.1

Verified by `scripts/audit_v01_coverage.py`; full report at `ablations/v01_coverage_audit.txt`. v0.1 had no task family for any of the four behaviors below; v0.2 additions are the coverage fix (some remain roadmap if not yet in `held_out`).

- 1. Thread-level coherence.** v0.1 tasks grade **single-shot** outreach; **no** multi-turn thread is in the task inventory. **v0.2 addition:** a **thread-coherence** partition (prior email + reply + follow-up) with rubric checks for contradictions across turns.
- 2. Pricing scope violations.** v0.1 does **not** include tasks whose pass/fail hinges on **TCV or quote band** vs `pricing_sheet.md`. **v0.2 addition:** tasks with explicit **allowed band** metadata plus **regex / structured extract** on quoted numbers in the body.
- 3. LinkedIn-roast / reputational tone.** v0.1 has **no** probe for “would this get screenshotted as bad outreach” from the style guide. **v0.2 addition:** a **reputational-tone** partition of 6–10 tasks presenting known-style-guide-violating drafts as candidates (new task family, not only a scorer change), graded by an `llm_judge_hook` in `scoring_evaluator.py` with style-guide context (non-deterministic but targeted).
- 4. Private referral / champion context.** Briefs are built from **public** signals and **redacted** case-study patterns; v0.1 includes **no** tasks where the correct motion depends on **non-public relationship facts** (e.g. dormant champion, internal blocker, warm intro via a named ally). **v0.2 addition:** a **synthetic private-context** field in the brief JSON and tasks that fail if the draft mishandles that metadata.

Public-Signal Lossiness

Ground-truth faithfulness (existing tasks): Rubric fields and scorer inputs are **proxies** for a live Tenacious motion — **public hiring and layoffs text**, **funding headlines**, and **redacted case-study-style** briefs. **Mechanism:** those signals can **lag** org reality or **omit named-account nuance** that a rep would use in production. The bench still scores “correct” relative to the **provided** brief, so an output that **parrots stale public facts** can **match reference** while **underperforming** in a live account. **Headline impact:** **Delta A/B compare**

candidate vs reference under the same brief, so the **relative lift** is internally valid; **absolute** “would this win the deal” quality can still be **optimistic** vs production because the **grounding truth is public-signal-limited**.

Authoring lossiness (separate issue): All 47 held-out tasks use **stub** signal_line text matching Ref=tbv02-... (none empty; see signal_line_distribution in inference_cost_estimate.json). The evaluator's signal_grounding_check scores **placeholder** lines weakly (no realistic funding/hiring tokens to ground against) — so grounding checks are **not representative of production signal richness** on this split. This is a **dataset construction** issue, not a scoring bug. The old **23.4%** raw candidate-output pass rate understates deterministic performance on tasks with **realistic** signal lines. **Impact on Delta A/B:** none — the preference-accuracy metric compares candidate_output vs ground_truth_output under the same rubric; stub signal lines affect both sides equally. **v0.2 fix (roadmap):** replace stub lines with specific plausible signals (funding amount, date, role count) and re-run the scorer; this will activate signal_grounding_check and produce a more informative absolute pass rate.

Honest Unresolved Failure: ICP Misclassification

The trained judge correctly prefers the reference output on only 2 of 6 icp_misclassification held-out tasks (33.3%, mean margin -0.26). **Unresolved training-process failure:** after SimPO tuning, the LoRA adapter still assigns the **wrong log-probability ordering** on many dual-segment ICP pairs, so the trained judge systematically prefers the weaker candidate when the decision requires weighing two segment rules at once. The failure pattern is narrow but concrete: this slice is the only category below 100%, and its negative mean margin shows convergence to an unstable or reversed preference boundary rather than mere random error. The next training step is to add targeted contrast pairs that isolate segment-rule tradeoffs and then re-check whether the adapted model can push this slice to **at least 80%** without regressing the categories already at 100%.

Kill-Switch Condition

Production-observable gate (primary — no held-out re-run required): On **weekly** random samples of $n \geq 25$ critic decisions (stratify approve/reject), **disable** the critic and revert to the **prompt-only judge on the same backbone** if **blind human reviewers** mark $\geq 35\%$ of sampled **rejections** as **clear false positives for two consecutive weeks** — a single week at $\geq 35\%$ is treated as a **warning**, not an automatic rollback (binomial noise; see docs/kill_switch_calibration.md). **35%** is anchored near the worst held-out slice (~33% on ICP), so exceeding it for two consecutive weeks signals a collapse toward the known failure floor. Optionally track **human override rate** (sends approved despite critic reject): **two consecutive weeks** above **30%** triggers the same **disable + revert** pending investigation. **30%** is calibrated against the **+42.6pp Delta B lift** — at a 30% override rate, reps are approving enough flagged outputs to degrade the deployed lift toward the 48.9% prompt-only baseline, at which point the critic adds send-time friction without net filtering value. **Pilot thresholds** — recalibrate after ≥ 4 weeks of audits ($n \geq 25/\text{week}$).

Benchmark-regression gate (secondary — triggered only after a model, adapter, or judge-prompt change, not as a routine production check): After any such change, recheck with:

```
python3 scoring_evaluator.py --task-file tenacious_bench_v0.2/held_out/tasks.jsonl
```

Roll back if **icp_misclassification** preference accuracy falls **below 60%** (floor above the observed held-out **33.3% (2/6)**) or if any category currently at **100%** drops **below 80%** (**80%** = material break from perfect slice scores on gap_overclaiming, tone_drift, etc.). Compare against ablations/ablation_results.json as the reference baseline.